

Herbicide Trial Manager - Complete App Documentation

Project Overview

Name: Herbicide Trial Manager

Type: Professional mobile-first agricultural research platform

Purpose: Track, evaluate, and report on herbicide (and other agri-input) efficacy trials

Platform: Web App, PWA (Progressive Web App), Android APK via Capacitor

Repository: <https://github.com/pedduhudga/Miklens-herbicide-trial-manager-6>

Tech Stack & Architecture

Frontend

- **React 19.2.6** - UI framework
- **Vite 8.0.12** - Build tool and dev server
- **React Router DOM 7.15.1** - Client-side routing with `HashRouter`
- **Tailwind CSS v4.3.0** - Utility-first styling framework
- **Lucide React 1.17.0** - Icon library

Backend & Data

- **Google Apps Script** - Primary backend via Google Sheets API
- **Firebase 12.13.0** - Alternative backend for authentication and Firestore database
- **IndexedDB** - Client-side offline storage via custom `offlineDB.js`

Maps & Visualization

- **Leaflet 1.9.4** - Interactive maps
- **React Leaflet 5.0.0** - React wrapper for Leaflet
- **Vis Network 10.1.0** - Network graph visualization
- **Chart.js 4.5.1** - Data visualization and charting

Native & Hybrid

- **Capacitor 8.3.4** - Cross-platform native app framework
 - Android 8.3.4
 - Camera 8.2.0
 - Filesystem 8.1.2
 - Network 8.0.1
 - Splash Screen 8.0.1

- Status Bar 8.0.2

AI & Image Processing

- **Google Generative AI (@google/genai 2.6.0)** - AI-powered photo analysis
- **Cropper.js 2.1.1** - Image cropping tool
- **React Cropper 2.3.3** - React wrapper for cropper
- **QRCode 1.5.4** - QR code generation
- **jsQR 1.4.0** - QR code scanning

Document & Report Generation

- **jsPDF 4.2.1** - PDF generation
- **jsPDF AutoTable 5.0.8** - PDF table generation
- **html-docx-js 0.3.1** - Word document (.docx) generation
- **PPTXGENJS 4.0.1** - PowerPoint presentation generation
- **JSZip 3.10.1** - ZIP archive handling
- **File Saver 2.0.5** - Client-side file downloads

Statistics & Analysis

- **jStat 1.9.6** - Statistical calculations
-

Multi-Platform Deployment

1. Web Application (GitHub Pages)

- **Deployment:** Automatic via GitHub Actions on `main` branch push
- **Routing:** `HashRouter` for static hosting compatibility
- **Asset Loading:** Relative paths with `base: './'` in `vite.config.js`
- **URL Format:** `https://pedduhudga.github.io/Miklens-herbicide-trial-manager-6/#/`

2. Progressive Web App (PWA)

- **Manifest:** `public/manifest.json`
- **Service Worker:** `public/sw.js`
- **Installable:** Yes, on mobile and desktop browsers
- **Offline Support:** Via IndexedDB for data caching
- **PWA Features:**
 - Standalone display mode
 - App icon (SVG favicon)

- Theme color: #059669 (Emerald)
- Meta tags for iOS and Android

3. Native Android App (Capacitor)

- **Build Command:** npm run build && npx cap sync android && npx cap open android
 - **Configuration:** capacitor.config.json
 - **CLI:** @capacitor/cli 8.3.4
 - **Output:** Android Studio opens to build APK/AAB
 - **APK Signing:** Configure in Android Studio
 - **Instructions:** See MOBILE_BUILD_INSTRUCTIONS.md
-

Project Structure

```
Miklens-herbicide-trial-manager-6/
├─ src/
│   ├─ App.jsx           # Main app component with routing
│   ├─ App.css           # Global app styles
│   ├─ main.jsx          # React entry point
│   ├─ index.css         # Global CSS
│   │
│   └─ pages/            # Full-page components (27 pages)
│       ├─ Dashboard.jsx # Home/main dashboard
│       ├─ Trials.jsx     # Trial management (4487 lines)
│       └─ LargeScaleTrials.jsx # Large-scale trial management (3370
lines)
│           ├─ Projects.jsx      # Project management (1448 lines)
│           ├─ Formulations.jsx  # Formulation library
│           ├─ Ingredients.jsx   # Ingredient management
│           ├─ Organisations.jsx # Organization/user management
│           ├─ PlotScanner.jsx   # QR code plot scanner
│           ├─ FieldMap.jsx      # Interactive field mapping
│           ├─ Reports.jsx       # Report generation
│           ├─ Analytics.jsx     # Advanced analytics
│           ├─ Statistics.jsx    # Statistical analysis
│           ├─ DoseResponse.jsx  # Dose-response curve analysis
│           ├─ ResistanceTracker.jsx # Herbicide resistance tracking
│           ├─ AIAssistant.jsx   # AI-powered image analysis
│           ├─ SmartSearch.jsx   # Advanced search functionality
│           ├─ DataManagement.jsx # Data import/export
│           ├─ Alerts.jsx        # Alert/notification system
│           ├─ Settings.jsx      # App configuration
│           └─ UserManagement.jsx # User/role management
```

			└─ CompareTrials.jsx	# Side-by-side trial comparison
			└─ Setup.jsx	# Initial configuration
			└─ Login.jsx	# Authentication
			└─ MigrationTool.jsx	# Data migration
			└─ LiveTrialPage.jsx	# Public live trial QR page
			└─ CategorySelector.jsx	# Multi-product category switcher
			└─ PlaceholderPage.jsx	# Placeholder for development
			└─ components/	# Reusable UI components (19
			components)	
			└─ Sidebar.jsx	# Navigation sidebar (266 lines)
			└─ BottomNav.jsx	# Mobile bottom navigation
			└─ TopBar.jsx	# Header/top navigation
			└─ TrialCard.jsx	# Trial card display (387 lines)
			└─ PlotMap.jsx	# Interactive plot/field map (537
			lines)	
			└─ GridWeedCoverTool.jsx	# Visual grid for weed cover (266
			lines)	
			└─ CameraCapture.jsx	# Camera/photo capture (275 lines)
			└─ QRScanner.jsx	# QR code scanner
			└─ CloudBackup.jsx	# Cloud backup UI (553 lines)
			└─ SprayAdvisor.jsx	# Spray advice component (390 lines)
			└─ SmartAlerts.jsx	# Alert notifications (341 lines)
			└─ SyncStatus.jsx	# Sync status indicator (322 lines)
			└─ CropperModal.jsx	# Image cropper modal (239 lines)
			└─ PhotoGallery.jsx	# Photo gallery viewer
			└─ VoiceInput.jsx	# Voice input component
			└─ Toast.jsx	# Toast notifications
			└─ Modal.jsx	# Generic modal dialog
			└─ ChartCard.jsx	# Chart container
			└─ LoadingOverlay.jsx	# Loading spinner overlay
			└─ hooks/	# Custom React hooks
			└─ useAppState.jsx	# Global app state management (199
			lines)	
			└─ useAuth.js	# Authentication hook (89 lines)
			└─ useSync.js	# Data sync hook (101 lines)
			└─ services/	# Business logic & APIs (19
			services)	
			└─ dataLayer.js	# Core data fetching (402 lines)
			└─ db.js	# Database abstraction (243 lines)
			└─ firebase.js	# Firebase initialization (95 lines)
			└─ firebaseAuth.js	# Firebase auth (173 lines)
			└─ firebaseDB.js	# Firebase Firestore operations (462
			lines)	
			└─ offlineDB.js	# IndexedDB offline storage (419

```

lines)
| | | └─ sync.js                # Data synchronization (452 lines)
| | | └─ syncManager.js         # Sync management (409 lines)
| | | └─ ai.js                  # AI/Gemini integration (21596
lines)
| | | └─ multiProviderAI.js     # Multi-AI provider support (590
lines)
| | | └─ alertsService.js       # Alert management (421 lines)
| | | └─ sprayAdvisor.js        # Spray recommendation logic (419
lines)
| | | └─ trialReports.js        # Report generation (1917 lines)
| | | └─ cloudBackup.js         # Cloud backup (560 lines)
| | | └─ weather.js            # Weather API integration (833
lines)
| | | └─ largeScaleService.js   # Large-scale trial logic (186
lines)
| | | └─ compareReports.js      # Trial comparison (346 lines)
| | | └─ mappingService.js      # Map/GIS services (520 lines)
| | | └─ sheetMirror.js         # Google Sheet mirroring (128 lines)
| | |
| | └─ utils/                   # Utility functions (14 utilities)
| | └─ categoryConfig.js        # Multi-category product config (448
lines)
| | | └─ analysisUtils.js       # Analysis calculations
| | | └─ auditUtils.js          # Audit trail utilities
| | | └─ coverUtils.js          # Weed cover calculation
| | | └─ dateUtils.js           # Date/time helpers
| | | └─ doseResponseUtils.js   # Dose-response analysis
| | | └─ exportUtils.js         # Data export functions
| | | └─ helpers.js             # General helpers
| | | └─ statsUtils.js          # Statistical calculations
| | | └─ voiceUtils.js          # Voice recognition utilities
| | | └─ weedUtils.js           # Weed species database
| | | └─ nativeCapabilities.js  # Native device features
| | | └─ perfUtils.js           # Performance utilities
| | | └─ aiConstants.js         # AI prompt templates
| | |
| | └─ assets/                  # Images, icons, etc.
|
└─ public/                      # Static assets
|   └─ favicon.svg              # App icon
|   └─ icons.svg                # Icon sprite sheet
|   └─ manifest.json            # PWA manifest
|   └─ sw.js                    # Service worker
|
└─ index.html                   # HTML entry point with meta tags
└─ vite.config.js               # Vite build configuration

```

— capacitor.config.json	# Capacitor native config
— package.json	# Dependencies and scripts
— eslint.config.js	# ESLint configuration
— README.md	# Project README
— MOBILE_BUILD_INSTRUCTIONS.md	# Android build guide
— Google sheet webapp script.txt	# Google Apps Script backend code

Core Features by Page

Dashboard (`Dashboard.jsx`)

- Overview of active trials
- Recent activity feed
- Key metrics summary
- Quick access to main features

Trials Management (`Trials.jsx`) - 4487 lines

- Create, edit, delete trials
- Trial card view with filtering and sorting
- Trial status tracking
- Efficacy data recording
- Multi-step trial wizard

Large-Scale Trials (`LargeScaleTrials.jsx`) - 3370 lines

- Manage field-scale trials with hierarchical structure
- Plot-level organization
- Visit/observation scheduling
- Bulk observation entry
- Geospatial plot mapping

Projects (`Projects.jsx`)

- Organize trials into projects
- Project-level reporting
- Multi-trial analytics

Formulations (`Formulations.jsx`)

- Formulation library management
- Add/edit/delete products

- Formulation-specific metadata
- Ingredient composition

Ingredients ((Ingredients.jsx))

- Manage active ingredients
- Chemical properties
- Regulatory information

Plot Scanner ((PlotScanner.jsx))

- QR code scanning for plot identification
- Real-time plot lookup
- Quick observation entry from scanned plot

Field Map ((FieldMap.jsx))

- Interactive Leaflet map integration
- Plot visualization
- Geospatial data management
- Map-based trial selection

Reports ((Reports.jsx))

- Multi-format report generation:
 - PDF trial reports
 - Word (.docx) documents
 - PowerPoint presentations (.pptx)
 - Excel spreadsheets (.xlsx)
- Customizable report templates
- Batch report generation

Analytics ((Analytics.jsx))

- Interactive charts and graphs
- Trial comparison visualizations
- Trend analysis

Statistics ((Statistics.jsx))

- Statistical analysis of trial data
- ANOVA, regression, hypothesis testing
- Statistical summaries

Dose-Response (`DoseResponse.jsx`)

- Dose-response curve modeling
- ED50 calculations
- Curve fitting algorithms

Resistance Tracker (`ResistanceTracker.jsx`)

- Track herbicide resistance trends
- Weed species resistance patterns
- Risk assessment

AI Assistant (`AIAssistant.jsx`)

- AI-powered photo analysis via Google Gemini
- Weed species identification
- Weed cover estimation
- Crop damage assessment
- Disease symptom recognition (fungicide mode)
- Pest identification (pesticide mode)

Smart Search (`SmartSearch.jsx`)

- Advanced search with filters
- Full-text search across trials
- Saved search queries

Data Management (`DataManagement.jsx`)

- Data import (CSV, Excel)
- Data export
- Data backup/restore
- Migration tools

Settings (`Settings.jsx`)

- Backend configuration (Firebase or Google Sheets)
- API key management
- User preferences
- App behavior settings

User Management ((UserManagement.jsx))

- User account management
- Role-based access control
- Permission assignment

Compare Trials ((CompareTrials.jsx))

- Side-by-side trial comparison
- Efficacy comparison charts
- Statistical comparison

Alerts ((Alerts.jsx))

- Alert notifications
- Trial status alerts
- Data quality warnings

Live Trial Page ((LiveTrialPage.jsx))

- Public QR code landing page
- No authentication required
- Real-time trial data display

Category Selector ((CategorySelector.jsx))

- Switch between product categories
- Category-specific UI/data

Setup ((Setup.jsx))

- Initial app configuration wizard
- Backend selection (Firebase or Google Sheets)
- API credentials setup

Login ((Login.jsx))

- User authentication
 - Support for Firebase Auth and custom credentials
 - Session management
-

Multi-Category Product Support

The app supports **5 product categories** with category-specific configuration:

1. Herbicide (Default - Emerald)

- **Primary Metric:** Weed Control Efficiency (WCE) = $(1 - \text{treated_cover}/\text{control_cover}) \times 100$
- **Target:** Weed species
- **Application Timings:** PRE, EPOST, POST, LPOST, SEQ
- **Key Fields:** Target weed species, weed growth stage, crop yield
- **Observations:** Weed cover %, weed species details
- **AI Features:** Weed identification, cover estimation, phytotoxicity
- **Special Tools:** Spray advisor, resistance tracker, grid weed cover tool

2. Fungicide (Indigo)

- **Primary Metric:** Disease Control Efficiency (DCE) = $(1 - \text{treated_severity}/\text{control_severity}) \times 100$
- **Target:** Crop disease
- **Application Timings:** Preventive, curative, eradicant, seed treatment, sequential
- **Key Fields:** Target disease, pathogen name, severity scale, inoculation method, FRAC group
- **Observations:** Disease severity %, incidence %, green leaf area, AUDPC, phytotoxicity
- **AI Features:** Disease symptom identification, severity estimation, leaf health assessment
- **Special Tools:** Spray advisor, disease finder

3. Pesticide (Red)

- **Primary Metric:** Pest Reduction Efficiency (PRE) = $(1 - \text{after_count}/\text{before_count}) \times 100$
- **Target:** Crop pest/insect
- **Application Timings:** Foliar, soil drench, seed treatment, granular, trunk injection, bait
- **Key Fields:** Target pest, pest species, initial pest density, life stage, IRAC group, PHI
- **Observations:** Pest count, damage rating, beneficial insect count, % mortality, knockdown rate
- **AI Features:** Insect identification, pest count estimation, damage assessment, beneficial detection
- **Special Tools:** Spray advisor, pest finder

4. Nutrition/Fertilizer (Amber)

- **Primary Metric:** Yield Improvement = $((\text{treated_yield} - \text{control_yield}) / \text{control_yield}) \times 100$
- **Target:** Nutrient type
- **Application Timings:** Basal, top dress, foliar, fertigation, seed coating, split
- **Key Fields:** Nutrient type, NPK ratio, micronutrients, application rate
- **Observations:** Crop growth stage, plant height, biomass, nutrient deficiency rating
- **AI Features:** Nutrient deficiency detection, crop health assessment
- **Special Tools:** Spray advisor

5. Biostimulant (Cyan)

- **Primary Metric:** Growth Response = $((\text{treated_param} - \text{control_param}) / \text{control_param}) \times 100$
- **Target:** Biostimulant type
- **Application Timings:** Seed coating, foliar, soil drench, fertigation
- **Key Fields:** Biostimulant type, active ingredients, concentration
- **Observations:** Root development, shoot vigor, stress tolerance, overall plant vigor
- **AI Features:** Plant vigor assessment, stress response detection

Category Config Location: `src/utils/categoryConfig.js` (448 lines)

Key Features:

- Dynamic UI/data model switching
 - Category-specific field schemas
 - Color-coded interfaces per category
 - Independent data collections per category
 - Custom efficacy calculations per category
 - Category-specific AI prompts
-

Data Management & Backend

Database Options

Option 1: Google Sheets + Apps Script

- **Setup:** Google Sheets as data store, Apps Script as API
- **Benefits:**
 - Free, easy to maintain
 - Familiar spreadsheet interface

- No server hosting needed
- **Data Structure:**
 - Sheets for each collection: trials, projects, formulations, ingredients, etc.
 - Apps Script (`Google sheet webapp script.txt`) handles CRUD operations
- **Services:**
 - `db.js` - Database abstraction layer
 - `dataLayer.js` - High-level data operations
 - `sheetMirror.js` - Sheet-specific operations

Option 2: Firebase (Recommended for scalability)

- **Setup:** Firebase Firestore for database, Firebase Auth for users
- **Benefits:**
 - Real-time data synchronization
 - Built-in authentication
 - Cloud hosting
 - Better scaling
- **Services:**
 - `firebase.js` - Firebase initialization
 - `firebaseAuth.js` - Authentication management
 - `firebaseDB.js` - Firestore operations

Data Sync & Offline Support

Offline-First Architecture:

- **IndexedDB Storage** (`offlineDB.js`) - Client-side caching
- **Sync Manager** (`syncManager.js`) - Background sync
- **Sync Hook** (`useSync.js`) - React hook for sync integration
- **Smart Sync** (`sync.js`) - Conflict resolution and data merging

Sync Features:

- Automatic background sync when online
- Queue operations during offline mode
- Conflict detection and resolution
- Sync status indicator component

Component Library

Layout Components

- **SideBar** - Navigation drawer with collapsible menu
- **BottomNav** - Mobile-friendly bottom navigation
- **TopBar** - Header with title and actions
- **Modal** - Reusable modal dialog
- **Toast** - Notification messages

Trial-Specific Components

- **TrialCard** - Trial summary card with metadata
- **PlotMap** - Interactive Leaflet map for plot visualization
- **GridWeedCoverTool** - Visual grid for estimating weed cover percentage

Data Capture

- **CameraCapture** - Mobile camera integration (Capacitor)
- **QRScanner** - QR code detection and reading
- **CropperModal** - Image cropping interface
- **VoiceInput** - Voice-to-text input

Analytics & Reporting

- **ChartCard** - Chart.js wrapper for data visualization
- **SprayAdvisor** - Spray recommendation component
- **SmartAlerts** - Alert notification system

System Components

- **CloudBackup** - Cloud backup UI
- **SyncStatus** - Real-time sync status indicator
- **PhotoGallery** - Photo gallery viewer
- **LoadingOverlay** - Loading spinner overlay

Services & Business Logic

Core Services

`dataLayer.js` (402 lines)

- High-level data fetching and caching
- `getAllData()` - Fetch all trials, projects, formulations, etc.

- Multi-category data retrieval
- Error handling and retry logic

`db.js` (243 lines)

- Database abstraction layer
- Routes calls to Firebase or Google Sheets
- Unified CRUD interface
- Backend-agnostic operations

`sync.js` (452 lines)

- Data synchronization logic
- Conflict detection
- Merge strategies
- Queue management

`syncManager.js` (409 lines)

- Sync orchestration
- Background sync scheduling
- Network detection
- Sync status tracking

AI & Analysis

`ai.js` (21,596 lines - Largest service)

- Google Generative AI (Gemini) integration
- Multi-prompt system for photo analysis
- Weed identification and cover estimation
- Disease symptom recognition
- Pest identification
- Crop damage assessment
- Phytotoxicity detection
- Cached analysis results
- Token usage tracking

`multiProviderAI.js` (590 lines)

- Support for multiple AI providers
- Provider abstraction layer
- Fallback mechanism

Specialized Services

`trialReports.js` (1917 lines)

- Report generation
- PDF export (jsPDF)
- Word document generation (.docx)
- PowerPoint creation (.pptx)
- Excel spreadsheet export
- Custom templates
- Report scheduling

`alertsService.js` (421 lines)

- Alert creation and management
- Alert notifications
- Alert filtering and display

`sprayAdvisor.js` (419 lines)

- Spray recommendation logic
- Weather-based recommendations
- Growth stage recommendations
- Historical efficacy analysis

`weather.js` (833 lines)

- Weather API integration
- Current weather data
- Forecast integration
- Spray timing recommendations

`cloudBackup.js` (560 lines)

- Cloud storage integration
- Backup scheduling
- Restore functionality

`largeScaleService.js` (186 lines)

- Large-scale trial logic
- Plot hierarchy management
- Visit scheduling

- Bulk operations

`compareReports.js` (346 lines)

- Trial comparison logic
- Statistical comparisons
- Efficacy comparisons

`mappingService.js` (520 lines)

- GIS/mapping utilities
 - Geospatial calculations
 - Plot positioning
 - Map layer management
-

Hooks (State Management)

Global State - `useAppState.jsx` (199 lines)

Context: `AppStateProvider`

State Properties:

```
javascript

{
  auth: { uid, username, password },
  settings: {
    firebaseEnabled,
    firebaseConfig: { apiKey, projectId, ... },
    scriptUrl, sheetId, folderId,
    ...
  },
  trials: [],
  projects: [],
  formulations: [],
  ingredients: [],
  organisations: [],
  blocks: [],
  activeCategory: 'herbicide',
  hasLoadedInitialData: false,
  isOnline: true,
  platformAdapter: { showToast, showLoading, renderSyncStatus }
}
```


- `updateState(updates)` - Merge state updates
- `getAppState()` - Get current state snapshot

Authentication - `useAuth.js` (89 lines)

- Determine if user is authenticated
- Handle login/logout
- Support Firebase Auth and custom credentials

Data Sync - `useSync.js` (101 lines)

- Background data synchronization
 - Auto-sync on interval
 - Triggered sync on app focus
 - Handles offline/online transitions
-

Utils & Helpers

Configuration

- `categoryConfig.js` (448 lines) - Multi-category product definitions (see Multi-Category section above)

Analysis & Calculations

- `analysisUtils.js` - Trial statistical analysis
- `doseResponseUtils.js` - Dose-response curve calculations
- `statsUtils.js` - General statistical functions
- `coverUtils.js` - Weed cover percentage calculations

Data Management

- `exportUtils.js` - CSV, Excel, PDF export functions
- `auditUtils.js` - Audit trail and change logging

Utilities

- `dateUtils.js` - Date/time formatting and calculations
- `helpers.js` - General-purpose helper functions
- `weedUtils.js` - Weed species database and utilities
- `voiceUtils.js` - Voice recognition and speech-to-text

- **nativeCapabilities.js** - Capacitor native feature detection
- **perfUtils.js** - Performance monitoring and optimization

AI

- **aiConstants.js** - AI prompt templates and constants
-

Authentication & Authorization

Firestore Authentication Flow

1. User enters Firebase credentials (email/password)
2. Stored in `state.auth.uid`
3. Firebase SDK authenticates user
4. Credentials passed to Firestore for authorization

Custom Google Sheets Authentication

1. User enters username/password
2. Stored in `state.auth.username` and `state.auth.password`
3. Passed to Apps Script for API authentication
4. Script validates credentials

Role-Based Access Control

- User roles stored in database
 - Permissions checked on page load
 - Feature visibility based on permissions
 - Admin functions restricted to admin users
-

Key Features Summary

Core Trial Management

- ✓ Create, edit, delete trials
- ✓ Multi-category product support
- ✓ Hierarchical trial structure (projects → trials → plots → visits → observations)
- ✓ Standard and large-scale trial modes
- ✓ Trial status tracking
- ✓ Custom field schemas per category

- ✓ QR code plot scanner
- ✓ Photo capture with cropping
- ✓ Voice input/dictation
- ✓ Grid-based weed cover estimation
- ✓ Offline data entry with sync

AI-Powered Analysis

- ✓ Weed species identification
- ✓ Weed cover estimation
- ✓ Crop damage assessment
- ✓ Disease symptom recognition
- ✓ Pest identification
- ✓ Phytotoxicity detection
- ✓ Cached analysis to reduce API calls

Reporting & Export

- ✓ PDF trial reports
- ✓ Word documents (.docx)
- ✓ PowerPoint presentations (.pptx)
- ✓ Excel spreadsheets (.xlsx)
- ✓ Custom report templates
- ✓ Batch report generation

Analytics & Insights

- ✓ Interactive charts (Chart.js)
- ✓ Statistical analysis (ANOVA, regression)
- ✓ Dose-response curve modeling
- ✓ Trial comparison
- ✓ Efficacy visualization
- ✓ Trend analysis

Field Operations

- ✓ Interactive field maps (Leaflet)
- ✓ Plot-level geospatial data
- ✓ Weather integration
- ✓ Spray timing advisor
- ✓ Crop growth stage tracking

- ✓ Weed resistance tracking
- ✓ Weed species database
- ✓ HRAC group classification
- ✓ Multiple spray timings (PRE, POST, etc.)

Fungicide-Specific

- ✓ Disease severity scales
- ✓ Inoculation tracking
- ✓ FRAC group classification
- ✓ AUDPC calculation

Pesticide-Specific

- ✓ Pest density tracking
- ✓ Beneficial insect monitoring
- ✓ IRAC group classification
- ✓ Knockdown rate assessment

System Features

- ✓ Offline-first with auto-sync
 - ✓ Real-time sync status
 - ✓ Cloud backup capability
 - ✓ Firebase or Google Sheets backend
 - ✓ PWA installable
 - ✓ Android APK native app
 - ✓ Role-based access control
 - ✓ Audit trail
 - ✓ Multi-language ready
-

Development Setup

Prerequisites

- Node.js 18+ and npm
- Git
- Android Studio (for APK builds)
- Google account (for Firebase or Google Sheets)

```
bash
```

```
git clone https://github.com/pedduhudga/Miklens-herbicide-trial-manager-6.git  
cd Miklens-herbicide-trial-manager-6  
npm install
```

Development Server

```
bash
```

```
npm run dev
```

- Starts Vite dev server at (`http://localhost:5173`)
- Hot module replacement enabled
- Hash-based routing ready for GitHub Pages

Build

```
bash
```

```
npm run build
```

- Outputs optimized build to (`dist/`)
- Ready for GitHub Pages deployment
- Ready for Capacitor sync

Android APK Build

```
bash
```

```
npm run build  
npx cap sync android  
npx cap open android
```

- Android Studio opens for final APK building
- See (`MOBILE_BUILD_INSTRUCTIONS.md`) for details

Code Quality

```
bash
```

```
npm run lint
```

- ESLint checks code style
- React hooks linting enabled

Configuration Files

- **Vite:** `vite.config.js` - Build settings, relative base path for GitHub Pages
 - **Capacitor:** `capacitor.config.json` - Native app configuration
 - **ESLint:** `eslint.config.js` - Linting rules
 - **Tailwind:** Built into Vite via `@tailwindcss/vite`
-

Database Schema Overview

Core Collections

Trials

```
{
  id, projectId, categoryId, name, description,
  cropCrop, cropVariety, location, area,
  startDate, endDate, status,
  designType (RCBD, CRD, etc.),
  replications, treatments,
  controlTreatment,
  formulationId, doseRate, adjuvant,
  applicationTiming, growthStage,
  weedSpecies (herbicide specific),
  diseaseTarget (fungicide specific),
  pestTarget (pesticide specific),
  observations: [{
    visitDate, daa, weedCover, weedDetails,
    efficacy, damageRating, ...categorySpecific
  }],
  createdBy, createdAt, modifiedDate
}
```

Projects

```
{
  id, categoryId, name, description,
  startDate, endDate,
  location, region,
  trialIds: [],
  createdBy, createdAt
}
```

Formulations

```
{
  id, categoryId, name, product,
  type (SC, WP, OD, etc.),
  modeOfAction,
  targetWeeds/Diseases/Pests,
  activeIngredients: [{ name, percentage }],
  registeredCrops: [],
  phiDays, safetyData,
  efficacyRemarks,
  createdAt
}
```

Ingredients

```
{
  id, categoryId, name, scientificName,
  chemicalClass, modeOfAction,
  properties: { solubility, toxicity, ... },
  regulatoryStatus,
  createdAt
}
```

Organisations

```
{
  id, name, type, location,
  users: [{ userId, role, permissions }],
  settings: { ...organizationSettings },
  createdAt
}
```

Performance Optimizations

1. **Data Caching:** `dataLayer.js` caches fetched data
 2. **AI Result Caching:** Analysis results cached to reduce API calls
 3. **Lazy Loading:** Pages loaded on demand via React Router
 4. **Code Splitting:** Vite handles automatic code splitting
 5. **Image Optimization:** Cropper tool for large image reduction
 6. **Offline Storage:** IndexedDB for local caching
 7. **Sync Batching:** Multiple updates batched in sync
 8. **Token Tracking:** AI API token usage monitored
-

Security Considerations

1. **API Keys:** Firebase and Google Sheets credentials stored in browser state
 2. **HTTPS Required:** All communications encrypted in production
 3. **Authentication:** Firebase Auth or custom credentials
 4. **Authorization:** Role-based access control on features
 5. **Data Validation:** Input validation before database operations
 6. **Audit Trail:** All changes logged with user and timestamp
 7. **Offline Data:** IndexedDB data accessible to browser JavaScript
-

Common Workflows

Creating a New Trial

1. Navigate to Trials or Projects
2. Click "New Trial"
3. Select category (herbicide, fungicide, etc.)
4. Fill trial metadata (crop, location, dates, etc.)
5. Add treatments and control
6. Define observation schedule
7. Save trial

Recording Trial Data

1. Go to trial detail page
2. Add new observation/visit
3. Record efficacy data (weed cover %, severity, etc.)
4. Optionally capture photos
5. Use AI to analyze photos if enabled

6. Save observation
7. Data syncs automatically when online

Generating Report

1. Navigate to Reports page
2. Select trials to include
3. Choose report format (PDF, Word, PowerPoint, Excel)
4. Customize report template
5. Generate
6. Download or export

Comparing Trials

1. Go to Compare Trials page
 2. Select 2+ trials to compare
 3. Choose comparison metrics
 4. View side-by-side efficacy data
 5. Compare statistical analyses
-

File Size & Complexity

Largest Services

1. `ai.js` - 21,596 lines (AI integration, multi-model support)
2. `trialReports.js` - 1,917 lines (Report generation)
3. `weather.js` - 833 lines (Weather API)

Largest Pages

1. `Trials.jsx` - 4,487 lines (Main trial CRUD interface)
2. `LargeScaleTrials.jsx` - 3,370 lines (Large-scale trial management)
3. `Projects.jsx` - 1,448 lines (Project management)

Largest Components

1. `CloudBackup.jsx` - 553 lines (Backup/sync UI)
2. `PlotMap.jsx` - 537 lines (Interactive mapping)
3. `SprayAdvisor.jsx` - 390 lines (Recommendations)

- ~30,171 lines in services
 - ~20,222 lines in pages
 - ~4,177 lines in components
 - ~389 lines in hooks
 - ~14+ utilities with shared functions
 - **Total App Size:** ~55,000+ lines of JavaScript
-

GitHub Deployment

The app auto-deploys to GitHub Pages using GitHub Actions:

1. **Trigger:** Push to `main` branch
2. **Build:** `npm run build` creates `/dist` folder
3. **Deploy:** GitHub Actions copies `/dist` to `gh-pages` branch
4. **URL:** <https://pedduhudga.github.io/Miklens-herbicide-trial-manager-6/>

Hash-based Routing: All routes use `#!/` syntax for static hosting compatibility

- `#!/` - Dashboard
 - `#!/trials` - Trials page
 - `#!/reports` - Reports page
 - `#!/settings` - Settings page
-

Next Steps for Development

To Extend App:

1. Add new product category → Update `categoryConfig.js`
2. Add new page → Create in `src/pages/`, add route in `App.jsx`
3. Add new component → Create in `src/components/`
4. Add new feature → Create service in `src/services/`
5. Add analytics → Use `Chart.js` wrapped in `ChartCard` component

To Debug:

1. Check browser console (F12)
2. Use Redux/React DevTools extension for state debugging
3. Check network tab for API calls

4. Check Application tab for IndexedDB data
5. Use `console.log()` for quick debugging

To Deploy:

1. Push changes to `main` branch
 2. GitHub Actions automatically builds and deploys
 3. Clear browser cache to see updates
 4. For APK: Run `npm run build && npx cap sync android && npx cap open android`
-

Contact & Support

Developer: Peddu (Pavan R)

Organization: Miklens Bio Pvt. Ltd.

Repository: <https://github.com/pedduhudga/Miklens-herbicide-trial-manager-6>

Last Updated: June 2026

App Version: 6.0

React Version: 19.2.6

Vite Version: 8.0.12

Deployment: GitHub Pages + Capacitor APK